



Informe TPO Base de datos II

Grupo N°10
Base de Datos II - Comisión S

Integrantes:

Javier Liu - 64332
Nicolas Rivas - 64292
Juan Ignacio Causse - 61105

Profesores:

Guillermo Rodriguez
Cecilia Rodriguez Babino

Decisiones de diseño:

Para migrar VetSalud a un nuevo sistema de persistencia NoSQL, se decidió que lo más apropiado es utilizar 2 bases de datos distintas. Como base de datos principal, se decidió utilizar MongoDB gracias a su gran versatilidad a la hora de almacenar datos, y realizar queries de forma sencilla.

Utilizamos MongoDB en todas las queries exceptuando 2 queries que involucran la administración de stock, debido a que se considera que un sistema de manejo de productos podría tener una frecuencia muy alta de queries por segundo, MongoDB no es apropiado para el manejo de este. Por ese motivo, se decidió utilizar Cassandra para el manejo de stock de la red de clínicas veterinarias.

Respecto a la implementación específica, se decidió containerizar todos los servicios utilizando contenedores de Docker, y utilizar DockerCompose para orquestar los servicios, redes, y puertos, gracias a su facilidad de uso.

Para la implementación de la api, se decidió utilizar javascript, debido a su sencilla integración con MongoDB. Y debido a su facilidad de uso, se decidió utilizar express para exponer las queries en http.

Para las queries de tipo PUT o POST donde se agregan datos, se decidió validar el input de forma estricta, obligando al usuario de la api a completar todos los parámetros con el tipo de dato adecuado, de lo contrario se informa al cliente de los datos inválidos.

Para la implementación del frontend, se decidió implementar una SPA utilizando sólo Vanilla Javascript. De esta forma, se puede implementar de forma rápida y sencilla una interfaz gráfica web que es expuesta por un servidor nginx.

Para la carga de datos en MongoDB, se implementó un script que crea todas las tablas imitando la estructura de los CSV. A su vez, se incluye una sección adicional en el script que realiza la definición de las vistas. De esta forma, se generan las colecciones que se muestran en la Figura 1.

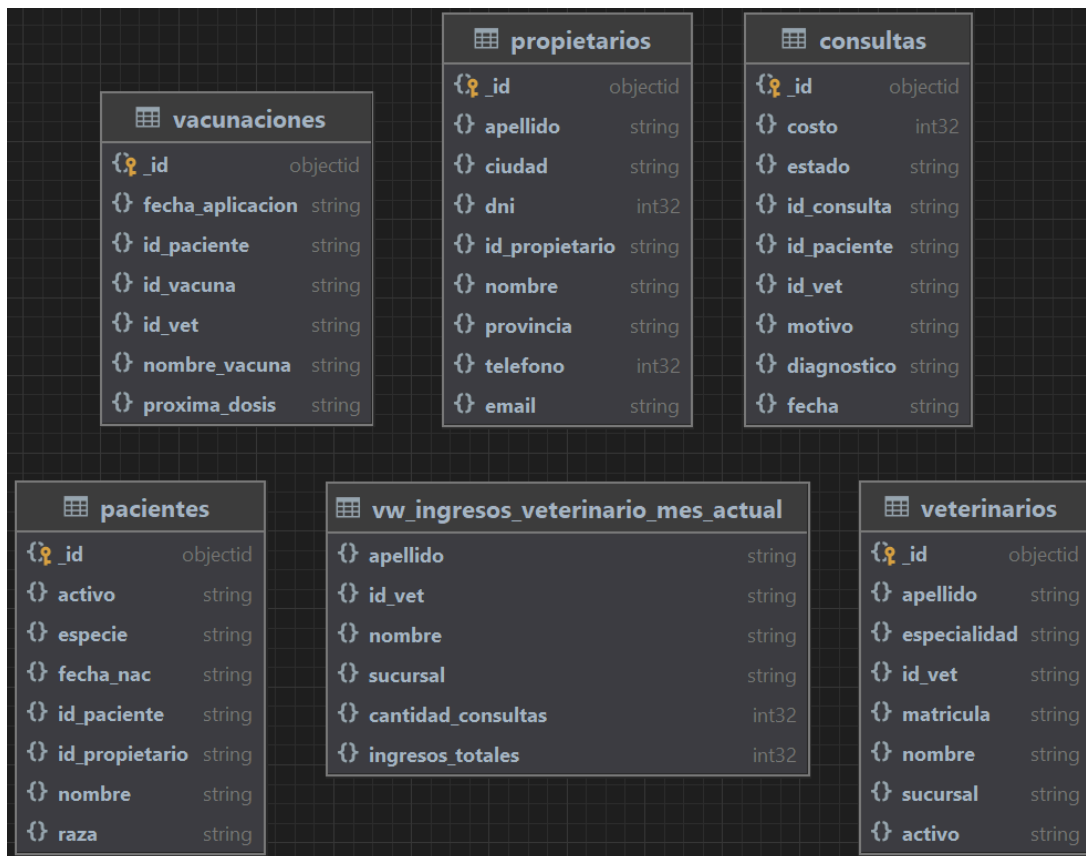


Figura 1: Esquema MongoDB.

Para el esquema de mongoDB, se decidió tener todos los datos normalizados como están en las tablas csv actualmente. Decidimos no hacer un esquema embebido y utilizar lookup porque las queries tienen combinaciones entre documentos muy variados. Además, no se incluyeron los datos de `stock_farmaceutico.csv`, dado que no lo utilizamos en ninguna consulta de mongoDB sino que en Cassandra.

Cuando se utiliza una gran cantidad de lookups, el rendimiento de MongoDB puede verse degradado bajo carga, aún así, se considera que esto no es un problema ya que no se espera una alta frecuencia de estas queries sobre MongoDB. Además, no embeber el esquema permite que las colecciones crezcan de forma independiente, disminuyendo el riesgo de superar el tamaño máximo de documento.

Para Cassandra, se intentó utilizar una filosofía similar, pero se encontraron algunas limitaciones que se tuvieron que resolver: Primero, es necesario definir el esquema de forma explícita, definiendo la key de cada tabla. También, ya que el contenedor de Cassandra no permite definir un script de inicialización, es necesario utilizar un SideCart container, con un archivo que se utiliza flag para saber si la base de datos ya fue inicializada.

stock_farmaceutico	
categoria	text
nombre	text
precio_unit	decimal
proveedor	text
unidades	int
vencimiento	date
id_producto	text

Figura 2: Esquema Cassandra.

Por la alta frecuencia de actualización mencionada anteriormente, se decidió que Cassandra realice toda la administración de stock, por lo que sólo fue necesario incluir este csv dentro de la base de datos.

Para que el stock se distribuya de forma equitativa entre los nodos, y no se generen nodos con exceso de carga, se decidió utilizar la ID del producto como partition key.

A su vez, ya que necesitamos mostrar los productos filtrados por bajo stock (stock menor a 50) se decidió crear un índice sobre el stock para poder realizar esta consulta. Pero, ya que un índice normal no permite realizar búsquedas con desigualdades, se decidió utilizar un SAI (Storage Attached Index), el cual permite realizar este tipo de búsquedas de forma eficiente (Bibliografía: [1]).

Ya que no hay ningún criterio de filtrado además del stock, Cassandra deberá consultar a todos los nodos para realizar la búsqueda de bajo stock. Se consideró crear buckets para agrupar los productos por cantidad de stock, pero esto podría generar una distribución asimétrica dentro del cluster, perjudicando nuevos casos de uso, por lo que se decidió mantener la implementación actual.

Por último, en la query de actualización masiva del stock se valida que la cantidad disponible no cambie múltiples veces durante la actualización, si en la misma query se solicita modificar 2 veces el stock del mismo producto, la query solo realiza una de las actualizaciones, y avisa al usuario que la actualización se realizó de forma parcial.

Ejemplos de resultados:

Una vez iniciado el proyecto utilizando `docker compose up` en la raíz del proyecto, es posible acceder a las queries mediante múltiples métodos, por ejemplo para la query2:

- Se puede acceder mediante [\[path del codespace\]/3000/query2](#) es decir, accediendo a [/query2](#) dentro del puerto 3000.

```
0:
  id_consulta: "CON004"
  id_paciente: "P004"
  id_vet: "V001"
  fecha: "2025-11-01"
  motivo: "Pérdida de apetito"
  diagnostico: "Estrés ambiental"
  costo: 3800
  estado: "Seguimiento"
  especialidad: "Clínica General"
  nombre: "Gabriela"
1:
  id_consulta: "CON005"
  id_paciente: "P005"
  id_vet: "V005"
  fecha: "2025-11-10"
  motivo: "Vómitos recurrentes"
  diagnostico: "Gastritis"
  costo: 5500
  estado: "Seguimiento"
  especialidad: "Clínica General"
  nombre: "Lorena"
2:
  id_consulta: "CON011"
  id_paciente: "P003"
  id_vet: "V008"
```

Figura 3: Resultado Query 2 accediendo desde la api.

- Se puede acceder mediante swagger, disponible en [\[path del codespace\]/\[3000\]/api-docs/#/default/get_query2](#) es decir, accediendo a [/api-docs/](#) en el puerto 3000.

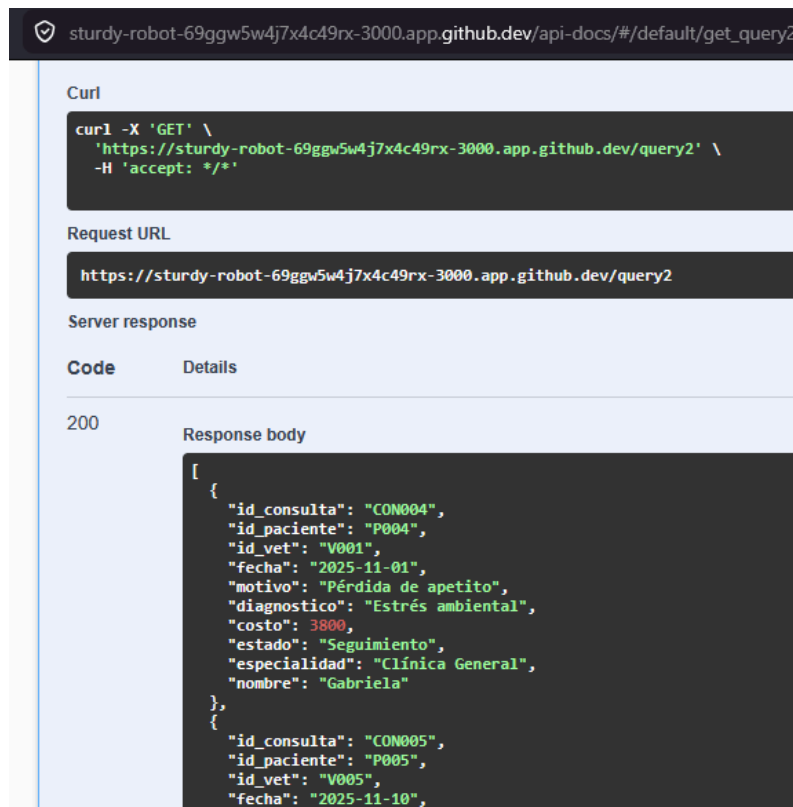


Figura 4: Resultado Query 2 accediendo desde Swagger.

- Se puede acceder mediante el frontend, disponible en [\[path del codespace\]/\[8080\]/#query2](#) es decir, accediendo al frontend desde el puerto 8080.

sturdy-robot-69ggw5w4j7x4c49rx-8080.app.github.dev/#query2

Query 2 — Consultas en seguimiento

Consultas con estado "Seguimiento" y datos del veterinario asignado

Resultados							
ID CONSULTA	ID VETERINARIO	ID PACIENTE	FECHA	MOTIVO	DIAGNÓSTICO	COSTO	ESTADO
CON004	V001	P004	2025-11-01	Pérdida de apetito	Estrés ambiental	\$ 3.800	Seguimiento
CON005	V005	P005	2025-11-10	Vómitos recurrentes	Gastritis	\$ 5.500	Seguimiento
CON011	V008	P003	2026-05-31	Dolor pata	Esguince	\$ 8.000	Seguimiento
CON013	V010	P005	2026-06-01	Vómitos	Gastritis	\$ 5.500	Seguimiento

Figura 5: Resultado Query 2 accediendo desde el frontend.

Bibliografía:

- Storage-attached indexing (SAI) concepts - Apache Cassandra
<https://cassandra.apache.org/doc/latest/cassandra/developing/cql/indexing/sai/sai-concepts.html>